skills_user_manual

Skills user manual (HXC-159 T-P8.01.3)

Revision: 1 · **Last modified**: 2026-09-23 · **Status**: current

This is the operator-facing manual for HelixCode's skills system — how to install a skill, where it is looked for, how it activates, and what each of the two consumers (`helix_code` and `submodules/helix_agent`) actually enforces once a skill matches. It complements, and does not duplicate:

- `docs/CAPABILITIES.md §4` — the HelixCode TUI's trigger-regex skill mechanism, with verified `file:line` wiring citations.
- `docs/CAPABILITIES.md §10.5` — the HelixAgent capability-enforcement model (`AllowedTools`, content-hash provenance, external tool-source registration, cross-consumer parity gate).
- `docs/guides/skills_directory_migration.md` — the historical narrative of why the directory model changed (HXC-159 Phase P5, T-P5.03 + T-P5.05).
- `docs/guides/skills_faq.md` — real questions this build surfaced, with real answers.

1. What a skill is

A skill is a single Markdown file with a YAML front-matter block declaring at minimum a `trigger` regular expression, plus a body — the text sent to the model when the trigger matches. Optional front-matter fields include `allowed-tools` (HelixAgent's capability-enforcement surface — see §4 below) and a description used by `/skills`-class introspection commands.

```
---
name: explain-arch
trigger: 'explain (?P<topic>\w+) architecture'
description: Render an architecture-explanation prompt for a
        subsystem.
---
Explain the architecture of the `{{topic}}` subsystem of
        HelixCode: its
package layout, its key types, and how it is wired into the rest
        of the
system.
```

The shipped example above lives at `.helix/skills/explain-arch.md` in this repository — try it with a prompt like `explain ensemble architecture` in the TUI, and `topic` resolves to `ensemble`.

2. Where a skill is discovered (four tiers, one precedence order)

Every HelixCode front-end (CLI, TUI, desktop) resolves the SAME ordered list of directories via `agent.AllSkillsDirs` (`internal/agent/skill_dirs.go:125`), lowest precedence first:

#	Tier	Path	Purpose
1	Legacy user	`$HOME/.helix/skills`	Pre-unification TUI/desktop location — read-compatibility only.
2	Canonical user	`os.UserConfigDir()/helixcode/skills`	The forward-looking, cross-platform-correct install target for new personal skills.
3	External	`.claude/skills` (or `$HELIXCODE_EXTERNAL_SKILLS_DIR`)	Skills *provisioned* by something other than hand-authoring — e.g. constitution-owned governance skills linked in by `scripts/register_skills.sh`.
4	Project	`.helix/skills` (per-project)	Highest precedence — a project skill overrides a same-named skill from any lower tier.

To install a personal skill that works everywhere: drop a `.md` file into tier 2, `os.UserConfigDir()/helixcode/skills` — on Linux this is typically `$XDG_CONFIG_HOME/helixcode/skills` or `~/.config/helixcode/skills`; on macOS `~/Library/Application Support/helixcode/skills`; on Windows `%AppData%/helixcode/skills`.

To install a project-specific skill: drop it into `.helix/skills/` at the project root — it will override any same-named skill from a lower tier for that project only.

A skill installed before HXC-159 Phase P5 (i.e. under the old `$HOME/.helix/skills` location for the TUI/desktop) is still found — that directory is tier 1 and remains scanned — but new installs should target tier 2. See `docs/guides/skills_directory_migration.md` for the full history.

3. How activation works (the TUI/CLI trigger model)

On every chat submit, the front-end runs `agent.DispatchSkill` against the loaded registry (`internal/agent/skill_activation.go:58`). Every loaded skill's `trigger` regex is matched against the raw prompt text; on the first match, the skill's rendered body — with named capture groups (like `{{topic}}` above) substituted — **replaces** the prompt sent to the model. The operator's original message stays visible in the on-screen chat history; only what the model receives changes.

If no trigger matches, the prompt is sent unmodified — skills are additive, never mandatory.

4. Capability enforcement (HelixAgent only)

HelixCode's TUI/CLI skill system (§3 above) is intentionally lightweight: it substitutes a prompt, nothing more. `submodules/helix_agent` layers a fuller **Agent Skills capability model** on top of the same on-disk file format:

- **`allowed-tools` front-matter** declares which tools a skill's rendered prompt may cause the agent to call. `SkillAuthorizer.Authorize` (`submodules/helix_agent/internal/skills/authorizer.go:56`) refuses a tool call a skill's non-empty `allowed-tools` does not name; an **empty** `allowed-tools` is a deliberate backward-compatible allow-all. Every decision is recorded to an audit log you can inspect via `AuditLog()`.
- **Content-hash provenance pinning** — a skill's origin can be pinned by content hash rather than a mutable file path, recorded on the capability manifest (`content_hash_provenance`, `submodules/helix_agent/internal/skillconformance/manifest.go:49`).
- **External tool-source registration** — a skill loaded from outside the built-in tier is exposed as a callable tool via `ToolRegistry.RegisterExternalToolSource` (`submodules/helix_agent/internal/services/tool_registry.go:159`).

helix_code's own TUI/CLI loader does NOT yet enforce `allowed-tools`. This is a real, tracked, currently-open gap surfaced by the cross-consumer parity gate (see §5) — see `docs/guides/skills_faq.md` for the operator-facing explanation.

5. The cross-consumer parity gate

Because `helix_code` and `helix_agent` each implement their own skill loader and their own capability-manifest generator, the two surfaces can silently drift apart — one consumer starts enforcing a capability the other one doesn't. `CM-CROSS-CONSUMER-PARITY` (`helix_code/cmd/skillparity`, `helix_code/internal/skillparity/parity.go:78`) diffs both consumers' manifests and fails on any **unexplained** asymmetry — a capability marked optional on both sides (via `OptionalCapability{Key, Reason}`, mutually acknowledged) is not a failure; a capability present on one side and silently absent from the other is.

Run it locally:

```
cd helix_code
go run ./cmd/skillconformance ... > /tmp/
        helix_code_report.json      # generate helix_code's
        report
# (helix_agent's own cmd/skillconformance generates the sibling
        report)
go run ./cmd/skillparity -a /tmp/helix_code_report.json -b /tmp/
        helix_agent_report.json
```

A non-zero exit means a real, currently-unmarked asymmetry exists — see the FAQ for the specific gap this build's own parity run found.

6. Sandboxed / third-party-tier execution — what's real today

If a skill declares itself in the `third-party` trust tier, HelixAgent gates its execution behind `EnforceSandboxPrecondition` (`submodules/skills/pkg/skills/sandbox_gate.go`), which refuses activation while `sandboxReady=false` — the honest current state, because the only shipped `IsolatedExecutor` implementation is `SkipIsolatedExecutor` (a deliberate no-op stand-in, not a real sandbox). **No skill in the `third-party` tier can execute today** until a real isolated executor lands upstream. This is by design (§11.4.6 no-guessing / §11.4.133 target-safety) — see the FAQ entry on the C5 sandbox precondition for the forensic detail.

7. Troubleshooting

Symptom	Likely cause	Fix
A skill I installed doesn't trigger	Front-matter `trigger` regex doesn't match, or	Verify the file is under one of the four tiers in §2; verify

Symptom	Likely cause	Fix
	the skill lives in a directory outside the four tiers	the regex with a quick `python3 -c "import re; print(re.search(r'<your regex>', '<your prompt>'))"`
Two front-ends see different skills	You installed under the pre-P5 `$HOME/.helix/skills` path expecting only the CLI to see it, or vice versa	All four tiers are now scanned by every front-end (§2); a genuinely front-end-specific skill is not a supported configuration — install at the project tier (`.helix/skills/`) if you need it scoped to one project
A `third-party`-tier skill refuses to run	The sandbox precondition gate (§6)	Expected — no real sandbox executor ships yet; use a `vendored` or `built-in` tier skill instead, or track the upstream sandbox work
`skillparity` fails in CI	A genuine, currently-unmarked capability asymmetry between the two consumers (§5)	See `docs/guides/skills_faq.md` for the specific known gap; if this is a NEW asymmetry, either close it or mark it `OptionalCapability` on BOTH sides with a reason

See also

- `docs/CAPABILITIES.md` — full HelixCode TUI capabilities guide (§4 Skills, §10.5 capability model).
- `docs/guides/skills_directory_migration.md` — the directory-unification history.
- `docs/guides/skills_faq.md` — frequently asked questions from this build.
- `constitution/docs/research/extensions/speckit_superpowers/implementation/` — the Bridge design corpus (constitution/HelixAgent skill-bridge architecture).